Kodys Foot Clinik v2.0 - Technical & Architecture Documentation

1. Executive Summary

This document provides a comprehensive overview of the **Kodys Foot Clinik (v2.0)** software stack, architecture, database schema, and operational flows. It is intended for software engineers, database administrators, and technical project managers to understand the existing system for maintenance, onboarding, and modernization (e.g., migrating from PyQt4 and addressing Windows-specific technical debt).

The Kodys software is a hybrid desktop-web application designed to interface with various foot and neurological testing hardware devices, process raw biomedical signals (such as ECG and HRV), generate clinical dashboards, and export structured medical PDF reports.

2. System Architecture

The project relies on a **Client-Server on Desktop** pattern, encapsulating a full web application stack within a localized desktop environment to seamlessly communicate with physical hardware.

2.1 High-Level Architecture

1. **Desktop Wrapper layer (`application_code/run.py`)**: A GUI executable utilizing Chromium Embedded Framework (CEF) and PyQt. It handles window management, local port binding, and raw serial communication with external USB/Serial devices.
2. **Web Backend (`Django 5.0.6`)**: An embedded web server that runs locally (`127.0.0.1:8000`). It is responsible for user authentication, routing, database transactions, and signal processing.
3. **Frontend Layer (HTML/CSS/JS)**: Rendered inside the desktop wrapper. Built with Django Templates, W3.css, and jQuery.
4. **Data Processing Engine**: Utilizes scientific Python libraries (`numpy` , `scipy` , `heartpy` , `pyhrv`) inside Django API views to transform raw device telemetry into structured clinical metrics.

2.2 Component Flow Diagram

- **Hardware Devices** -> (Serial RS232/USB) -> **Desktop Wrapper (`pyserial`)**
- **Desktop Wrapper** -> (JSON/AJAX POST) -> **Django Backend (REST/Views)**
- **Django Backend** -> (SciPy/NumPy Processing) -> **SQLite Database** & **wkhtmltopdf Engine**
- **Django Backend** -> (HTML/JS) -> **Frontend (CEF Browser UI)**

---

3. Technology Stack

3.1 Backend & Middleware

- **Core Framework**: Django 5.0.6 (Upgraded from 1.11.x)
- **Language Environment**: Python 3.14.3
- **Database**: SQLite3 (`db.sqlite3`)
- **Mathematical & Data Science Packages**:
  - `numpy` (v2.0.0): Vectorized operations and signal manipulation (e.g., NumPy trapezoidal integrals).
  - `scipy` : Signal filtering, integration, and noise reduction (50Hz filters).
  - `pandas` : Data structure handling.

- `heartpy` & `pyhrv`: Cardiac Autonomic Neuropathy (CAN) analysis, ECG processing, and Heart Rate Variability scoring.
- `peakutils`: Baseline estimation and peak detection in waveforms.

- **Security**: `cryptography` (Fernet symmetric encryption for sensitive data).

3.2 Frontend & UI

- **Templating**: Django Template Engine (HTML5).
- **Styling**: W3.CSS (Lightweight CSS framework) and custom `style.css` definitions.
- **Interactivity**: JavaScript (Vanilla & jQuery).
- **Graphing**: Canvas APIs and Chart.js for real-time visualization of Doppler waves and foot pressure graphs.

3.3 Desktop Integrations

- **GUI Framework**: `PyQt4` (Legacy) mapping to Chromium Embedded Framework (CEF).
- **Hardware Interface**: `pyserial` for interacting with `/dev/cu.usbserial-*` or `COM` ports.
- **Reporting**: `wkhtmltopdf` command-line tool invoked via subprocess to render HTML templates into PDF.

---

4. Database Schema Structure

The application employs a robust, relational schema built on Django's ORM.

4.1 Master Catalog & Configuration

- **`MA_APPLICATION` & `MA_APPLICATION_SETTINGS`**: Core app configuration, software version, hospital details, and dynamic font/UI settings.
- **`MA_MEDICALAPPS`**: The master registry of all hardware modules (APP-01 through APP-07).
- **`MA_MEDICALTESTMASTER` & `MA_MEDICALTESTFIELDS`**: Defines exactly which fields and instructions are associated with each test type.

4.2 Security, Access & Profiles

- **`auth_user`**: Standard Django authentication table mapping logins.
- **`TX_ACCOUNT`**: Hospital Profile data (Name, Logo, Phone, Address).
- **`MA_ACCOUNTROLE` & `TX_ACCOUNTUSER`**: Access control lists (Admin, Manager, Member) and their permissions.

4.3 Patient & Provider Management

- **`TX_HCP` (Healthcare Providers)**: Stores profiles for Doctors and Examiners, including specializations and unique UID trackers.
- **`TX_PATIENTS`**: Master patient indices. Stores demography (Age, Sex, UID), contact info, and vitals (Height, Weight).

4.4 Clinical Test Execution & Records

- **`TX_MEDICALTESTS`**: The central linking table. Maps a specific `Patient` to a `Test Type`, overseen by a `Doctor` and performed by an `Examiner`. Tracks the timestamp and status.
- **`TX_MEDICALTESTENTRIES`**: The granular data store. Contains the key-value pairs of raw numerical scores, boolean flags, or arrays resulting from a specific test run.
- **`TX_MEDICALTESTREPORTS`**: Stores paths to the generated visual graphs, processed data sets, and the finalized PDF document for physician review.

---

5. Medical Hardware Modules (App Codes)

The system supports specific clinical modules that correspond to individual routing endpoints and distinct hardware machines:

1. **APP-01 - Doppler Test**: Maps Arterial/Venous blood flow in the foot. Uses graphical representation to plot vascular health over time.
2. **APP-02 - Biothezi VPT (Foot & Hand)**: Vibration Perception Threshold test. Measures nerve sensitivity and diabetic neuropathy using variable voltage.
3. **APP-03 - Biothezi VPT Ultra**: Advanced quantitative sensory testing for neuropathy tracking.
4. **APP-04 - Thermocool**: Measures thermal perception thresholds (Hot/Cold mapping on the plantar surface).
5. **APP-05 - Podo I Mat**: Static and dynamic plantar pressure mapping via a sensor mat.
6. **APP-06 - Podo T Map**: Thermal temperature gradient mapping of the foot to identify pre-ulcerative hot spots.
7. **APP-07 - Kodys CAN (Cardiac Autonomic Neuropathy)**: Complete cardiovascular evaluation module processing ECG data to score Parasympathetic and Sympathetic nervous system health (HRV mapping).

---

6. Request Lifecycle & Execution Flow

1. **Initialization (`run.py`)**:
   - Application boots.
   - It executes `manage.py runserver` in a secondary process.
   - A desktop window initializes pointing to `http://127.0.0.1:8000/signin/` .
2. **Clinical Session Setup (`views.py`)**:
   - A Doctor/Examiner signs in.
   - Navigates through the dashboard to select a test module (`APP-01` to `APP-07`).
   - A `Patient` is searched/selected or created.
3. **Hardware Interaction (`run.py` & `app_api.py`)**:
   - User triggers "Start Test" on UI.
   - The desktop background thread queries the active `Serial` port.
   - Received raw telemetry is posted synchronously via AJAX to a Django endpoint.
4. **Signal Processing & Interpretation**:
   - Inside `app_api.py` , arrays of signal data are ingested.
   - E.g., for `APP-07` (CAN), `scipy.signal.butter` filters baseline wandering; `pyhrv` derives the R-R intervals from the ECG strip.
   - Results are converted into clinical grades (Normal, Early, Definite Neuropathy).
5. **Persistence & Export**:
   - Extracted features and clinical notes are logged to `TX_MEDICALTESTENTRIES` .
   - `wkhtmltopdf` is invoked to bundle the data template and graphical canvas snapshots into a localized PDF inside `/app_assets/DATA/reports/` .

---

7. Migration & Modernization Roadmap (Technical Debt)

The current implementation has accumulated technical debt that requires refactoring for cross-platform stability (macOS/Linux) and future maintainability:

1. **PyQt4 / Python 3.14 Incompatibility**:
    - `run.py` relies on `PyQt4`, a framework obsolete since Python 3.5.
    - *Recommendation:* Rewrite the desktop shell using `PyQt6` / `PySide6` or encapsulate the UI in an `Electron.js` container that bundles the Python runtime environment.
2. **OS Hardcoded Constraints (Windows Dependencies)**:
    - Module `winreg` is being imported for Windows registry checks.
    - `REPORT_TO_PDF_PATH` relies on Windows path styles (`\\wkhtmltopdf\\bin\\wkhtmltopdf.exe`).
    - *Recommendation:* Use Python's `os` and `pathlib` for dynamic pathing, and platform checks (`sys.platform`) before executing OS-specific registry bindings.
3. **Scientific Library Deprecations**:
    - `numpy 2.0` deprecated top-level features like `numpy.trapz`.
    - `scipy.integrate.simps` was removed in favor of `scipy.integrate.simpson`.
    - *Recommendation:* While monkey-patching exists in `app_api.py`, long-term maintainability dictates updating the core implementations of peak detection algorithms to use standard updated API signatures.